

AUTOMATIZACIÓN Y DESPLIEGUE DE UNA APLICACIÓN WEB CON KUBERNETES, K3S Y ANSIBLE



Universidad
Francisco de
Vitoria

UFV Madrid

PABLO PÉREZ HERRERO

**2º CURSO ADMINISTRACIÓN DE SISTEMAS INFORMÁTICOS EN
RED**

INDICE

1.	ABSTRACT	4
2.	RESUMEN Y OBJETIVOS	4
3.	ANTECEDENTES	5
4.	ANÁLISIS Y ESPECIFICACIÓN DE REQUISITOS	5
5.	PROPUESTA DE SOLUCIÓN	5
6.	PLAN DE TRABAJO	6
7.	DESARROLLO DE LA SOLUCIÓN	6
8.	DESPLIEGUE E INSTALACIÓN.....	7
8.1	CREACIÓN DE MÁQUINA VIRTUAL MASTER EN VIRTUAL BOX Y WORKERS	7
8.2	CREACIÓN DE MÁQUINA VIRTUAL MASTER EN VIRTUAL BOX Y WORKERS	7
9.	EVOLUCIÓN Y TRABAJO FUTURO	23
10.	BIBLIOGRAFÍA	23

1. ABSTRACT

This Project studies the design and implementation of automated deployment platforms using modern DevOps methods and container-oriented technologies. The solution is based on a lightweight Kubernetes distribution (k3s) deployed over a multi-node cluster and fully provisioned through Ansible, which enables scalation on the infrastructure. The application is based on Python, with Flask and SQLite containerized with Docker and orchestrated through Kubernetes objects as Deployments, Services, Secrets, and Ingress. The objective of this project is to provide a reliable, automated and maintainable environment that reflect real-world practices used on IT operations.

2. RESUMEN Y OBJETIVOS

El objetivo del proyecto es presentar una solución a la necesidad de alta disponibilidad de la mayoría de aplicaciones Web. Lo conseguiremos mediante una distribución ligera de Kubernetes (k3s) y Ansible, aunque esta parte es simplemente a nivel local, como demostración. El proyecto constará de dos partes bien definidas:

- Despliegue LOCAL de una app de 3 clúster balanceados, desplegándolos y configurándolos de forma automatizada para su escalabilidad. Comprobaremos y demostraremos como, con menos clusters, la disponibilidad es mucho mejor.
- Mismo despliegue pero en AWS, con prácticas DevOps como CI/CD, sustituyendo k3s por la misma aplicación de Amazon (EKS), igual con la base de datos que en este caso será Amazon RDS en vez de SQLite. Todo esto se implementaría a través de un pipeline CI/CD, con GitHub Actions o GitLab, que automatice todo el proceso, desde que se sube el código hasta que se despliegue una nueva versión de la aplicación.

3. ANTECEDENTES

Esta solución, aplicada en prácticamente cualquier despliegue de aplicación hoy en día (por supuesto, en el mundo profesional, son despliegues mucho más grandes y sofisticados), surge de la necesidad de despliegues automáticos y alta disponibilidad. Desplegar una aplicación manualmente, una vez por cada nodo de balanceo, es algo terriblemente ineficiente y que puede dar lugar a múltiples errores de todo tipo; cada paso manual es una oportunidad para crear incoherencias e inconsistencias entre las distintas fases del despliegue.

Cada una de las aplicaciones que se usan en el despliegue cubren una necesidad:

- **Docker:** Garantiza que tu aplicación web se despliegue siempre de la misma manera, dentro de un container, y siguiendo exactamente el mismo proceso, con la ventaja de que es una mini-virtualización securizada y aislada. Hace que funcione igual en cualquier sistema.
- **Kubernetes:** Permite que cada pod acceda a otro sin conocer su IP real, balancea los pods y realmente siempre estás accediendo al mismo servicio aunque sean diferentes pods. Además, incluye configuración SQL de forma nativa, lo cual facilita muchísimo el despliegue a nivel global, con el extra de tener Traefik (Ingress), que se puede usar para el reverse proxy, permitiendo acceder a la app desde fuera del cluster de forma segura. En nuestro caso, en el despliegue local, usaremos k3s, que es una versión ligera de Kubernetes.
- **Ansible:** Evita que tengamos que realizar configuraciones manuales en los servidores, asegura que todos los pods estén idénticamente configurados en el clúster y puede volver a levantar el mismo servicio de forma rápida y eficaz. Instala todas las dependencias, configura SSH para que el pod master hable con los workers, configura IPs, firewalls...

4. ANÁLISIS Y ESPECIFICACIÓN DE REQUISITOS

5. PROPUESTA DE SOLUCIÓN

6. PLAN DE TRABAJO

El plan de trabajo será el siguiente:

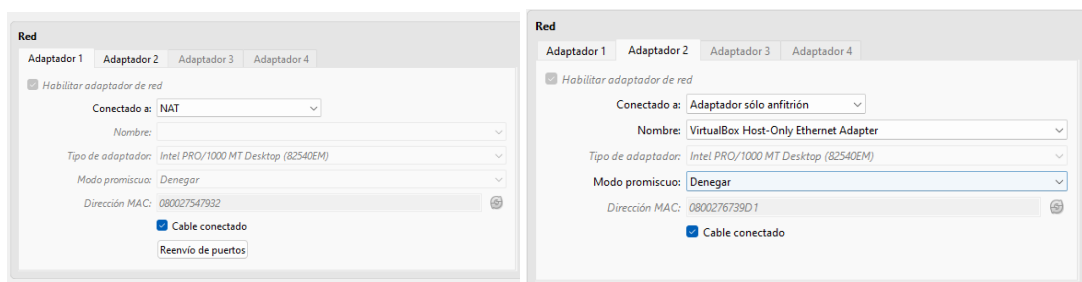
1. Crear un repositorio GIT y su correspondiente GitHub, para llevar una cronología del proyecto y tener una copia online, desacoplada del local (por si acaso).
2. Instalación y configuración de Máquinas virtuales. Creamos 3 máquinas virtuales, un master y dos workers, y le damos IP en la misma red local. Al master le damos salida a internet. Los workers realmente no necesitarían para el funcionamiento del servicio, pero sí para instalar los paquetes. Configuramos también el SSH y la clave desde el master a los workers.
3. Creamos el Inventario Ansible, y aplicamos los roles a cada una de las máquinas.
4. Instalamos k3s de forma automatizada.
5. Desarrollamos la aplicación, el dockerfile y la imagen de la aplicación.
6. Desplegamos en Kubernetes (k3s).
7. Probamos su funcionamiento, su rendimiento y damos los ajustes finales.

7. DESARROLLO DE LA SOLUCIÓN

8. DESPLIEGUE E INSTALACIÓN

8.1 CREACIÓN DE MÁQUINA VIRTUAL MASTER EN VIRTUAL BOX Y WORKERS

Lo primero es crear la máquina virtual Master en VirtualBox, que será la referencia para los dos Workers. Elegiremos Ubuntu server como distro, ya que es ligera y fácil de instalar y usar. Es muy importante que la máquina tenga dos adaptadores de red, el primero “red NAT” para salida a internet y el segundo “Solo Anfitrión” con una red que hemos preconfigurado, siendo 192.168.56.0/24.



Procedemos con la instalación de Ubuntu Server, una vez hemos creado el usuario y lo hemos instalado, tenemos que cambiarle el hostname a k8s-master (a partir de ahora, cada vez que me refiera a k8s, es un slang para Kubernetes). Por último, modificamos el adaptador de red en0s8, que es el de la red local, con el netplan, poniéndole su IP como 192.168.56.10.

Teniendo ya configurado el Master, lo más fácil es clonarlo para los dos workers, reiniciar sus MAC, cambiarles sus hostnames¹ a k8s-worker1 y k8s-worker2, cambiarles sus IPs a la 11 y la 12, y añadir las tres máquinas virtuales a los hostnames, para que podamos verlas sin tener que poner sus IPs; y ya tendríamos nuestro entorno virtual de trabajo creado, en menos de 15 minutos.

8.2 CREACIÓN DE MÁQUINA VIRTUAL MASTER EN VIRTUAL BOX Y WORKERS

A partir de ahora, prácticamente todo se hace con el master. Tenemos que instalar Ansible en el master, con SSH pass para que las máquinas se puedan comunicar entre ellas de forma segura. Lo instalamos con los siguientes comandos:

¹ Hago hincapié en los hostnames, porque son cruciales a la hora de desplegar Kubernetes y Ansible. Si no están bien puestos, no va a funcionar nada y los pods no se van a ver entre ellos.

```
***: Command not found
blillo@k8s-master:~$ sudo apt update -y && sudo apt upgrade -y && sudo apt install -y ansible sshpass_
```

Una vez tenemos Ansible, el siguiente paso es crear una clave SSH (para que cada vez que haya conexión, no esté preguntando por ella) y enlazar los dos worker con la clave del maestro. Generamos la clave² con:

```
blillo@k8s-master:~$ ssh-keygen -t ed25519 -C "ansible-control"
```

Copiamos la clave criptográfica a todas las máquinas con:

```
blillo@k8s-master:~$ ssh-copy-id blillo@192.168.56.10 && ssh-copy-id 192.168.56.11 && ssh-copy-id 192.168.56.12
```

Comprobamos que podemos hacer SSH entre las máquinas sin contraseña, y ya podríamos pasar a la parte interesante, que es crear el inventario para que Ansible conecte las máquinas. Podría escribir a mano cada documento en Ubuntu Server, pero al no tener interfaz gráfica, es un poco tedioso.

Mi método es, crear los documentos en mi PC (Windows) con VScode, por ejemplo, crear toda la estructura de carpetas, subirlo a un repositorio en GitHub y desde la máquina virtual master hacer un “git clone” de todo el contenido, matando dos pájaros de un tiro: tengo un seguimiento del trabajo tanto en mi PC como en las máquinas virtuales, y me ahorro mucho tiempo a la hora de escribir código.

8.3 CONFIGURACIÓN INICIAL DE ANSIBLE PARA EL DESPLIEGUE

Ahora tenemos que configurar Ansible para que reconozca a cada una de las máquinas y le pueda hacer su despliegue correspondiente. Empezaremos por `ansible.cfg`, donde configuraremos el archivo de inventario, el intérprete de Python, desactivaremos la comprobación de clave SSH, para que no esté preguntando continuamente, y quitamos la opción de crear archivos `retry`, para que no los genere automáticamente Ansible.

Hay que reseñar, que en este punto del proyecto decidí añadir dos máquinas mas, una para la base de datos y otra para hacer backups. También, cambié las IP, para que crearan menos conflicto. Ahora las IP van desde la 192.168.56.101 hasta la 192.168.56.105.

El archivo `ansible.cfg` quedaría así:

```
[defaults]
inventory = inventory/hosts.ini
host_key_checking = False
retry_files_enabled = False
interpreter_python = auto_silent
```

² Ed25519 es un algoritmo de encriptación basado en curvas elípticas

Además, necesitamos un archivo pequeño, requirements.yml, para decirle a Ansible que dependencias vamos a necesitar

```
collections:
  - name: kubernetes.core
  - name: community.mysql
```

Se instalan con el siguiente comando

```
blillo@app:~/pablo-asir-final/ansible$ ansible-galaxy collection install -r requirements.yml
Starting galaxy collection install process
Nothing to do. All requested collections are already installed. If you want to reinstall them, consider using '--force'.
blillo@app:~/pablo-asir-final/ansible$
```

En este caso, parece que ya estaban instaladas con esta versión de ansible. Ahora, vamos con el archivo hosts.ini, que es el que le dice a Ansible cual es la IP de cada máquina y cuál es su función.

```
[master]
app-master ansible_host=192.168.56.101

[workers]
app-worker1 ansible_host=192.168.56.102
app-worker2 ansible_host=192.168.56.103

[db]
app-db ansible_host=192.168.56.104

[backup]
app-backup ansible_host=192.168.56.105

[all:vars]
ansible_user=blillo
ansible_python_interpreter=/usr/bin/python3
```

También dice cuál es el usuario con el que hacer SSH y cuál será el intérprete de Python para ciertas acciones que realiza Ansible. Por último, vamos a crear el archivo de las variables de entorno para Ansible. Aquí se definen muchas cosas, para que en los playbooks no haya que andar repitiendo continuamente. Una cosa importante, es que si esto se sube a github, hay que crear un .gitignore y meter este archivo ahí, ya que contiene claves secretas que no deben ser publicadas. En este caso, como es un laboratorio local, nos da un poco igual.

El archivo vars.yml quedaría así:

```
lan_cidr: "192.168.56.0/24"

master_ip: "192.168.56.101"
worker_ips:
  - "192.168.56.102"
  - "192.168.56.103"
```

```
db_ip: "192.168.56.104"
backup_ip: "192.168.56.105"

kubeconfig_path: "/etc/rancher/k3s/k3s.yaml"

# App
inc_namespace: "incidencias"
inc_app_name: "incidencias-app"
inc_replicas: "3"
inc_image: "blillo90/incidencias:latest"
inc_host: "incidencias.local"

# MariaDB (app-db)
inc_db_name: "incidencias"
inc_db_user: "inc_user"
inc_db_pass: "root"
inc_db_host: "192.168.56.104"
inc_db_port: "3306"

# Backups (app-backup)
backup_db_user: "backup_user"
backup_db_pass: "root"
backup_dir: "/opt/backups/mariadb"
backup_retention_days: "7"
backup_time: "02:00"
```

Se puede ver bastante claramente lo que se está definiendo. Algo interesante, es que he tenido que crearme una cuenta en DockerHub, para Dockerizar la imagen de mi App, ya que está hecha con Python y es necesario paquetizarla. Desde ahí, la traemos hasta nuestra máquina e indicamos en estas variables la ruta de la imagen de nuestra app.

8.4 PLAYBOOKS DE ANSIBLE

Vamos con los playbooks. El primero, aunque ya lo hemos hecho, se asegurará de que los hostnames de las máquinas están bien, y que el archivo `/etc/hosts` está bien configurado. Este será `05-hosts-lab.yml`.

IMPORTANTE: para que funcione, necesitamos que no haga falta meter la contraseña de nuestro usuario cada vez que queramos hacer `sudo`. El método rápido es cambiar el archivo de sudores, con los siguientes comandos:

```
sudo visudo
#Añadimos al final
blillo ALL=(ALL) NOPASSWD:ALL
```

Al lanzar el playbook, nos saldrá una confirmación de que todo está OK.

```
blillo@app:~/pablo-asir-final/ansible$ ansible-playbook playbooks/05-hosts-lab.yml

PLAY [Asegurar /etc/hosts del laboratorio] *****

TASK [Gathering Facts] *****
ok: [app-master]
ok: [app-worker2]
ok: [app-worker1]
ok: [app-backup]
ok: [app-db]

TASK [blockinfile] *****
changed: [app-master]
changed: [app-db]
changed: [app-backup]
changed: [app-worker2]
changed: [app-worker1]

PLAY RECAP *****
app-backup      : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
app-db          : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
app-master      : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
app-worker1     : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
app-worker2     : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

blillo@app:~/pablo-asir-final/ansible$
```

El siguiente playbook es el hardening de las máquinas. Hay bastantes cosas que hacer, entre ellas, instalar Fail2Ban, permitir SSH (puerto 22), eso en general. En particular, en el master, permitir 80 y 443 para el servidor web, 6443 para la base de datos, 8472 UDP y 10250 para kubernetes y Ansible entre nodos. En los workers el de la base de datos no hace falta, y en la base de datos, el 3306 que es el que usa mysql.

Empezamos por el playbook general, que será `10-hardening.yml`

```
- name: Hardening base (UFW + Fail2Ban)
  hosts: all
  become: true
  vars_files:
    - ../group_vars/all/vars.yml
  tasks:
    - apt: { update_cache: true }

    - apt:
```

```

    name: [ufw, fail2ban]
    state: present

- ufw: { direction: incoming, policy: deny }
- ufw: { direction: outgoing, policy: allow }

- ufw:
    rule: allow
    port: "22"
    proto: tcp
    src: "{{ lan_cidr }}"

- ufw: { state: enabled }

- service:
    name: fail2ban
    state: started
    enabled: true

```

Tras obtener el mensaje de confirmación, vamos con el específico de configuración de UFW para el master, 11-ufw-master.yml

```

- name: UFW master (k3s server + ingress)
  hosts: master
  become: true
  vars_files:
    - ../group_vars/all/vars.yml
  tasks:
    - ufw: { rule: allow, port: "80", proto: tcp, src: "{{ lan_cidr }}" }
    - ufw: { rule: allow, port: "443", proto: tcp, src: "{{ lan_cidr }}" }
  }
    - ufw: { rule: allow, port: "6443", proto: tcp, src: "{{ lan_cidr }}" }
  }
    - ufw: { rule: allow, port: "8472", proto: udp, src: "{{ lan_cidr }}" }
  }
    - ufw: { rule: allow, port: "10250", proto: tcp, src: "{{ lan_cidr }}" }
  }

```

Si comprobamos el status de UFW, vemos que está todo correcto.

```
blillo@app:~/pablo-asir-final/ansible$ sudo ufw status
Status: active

To Action From
--
22/tcp ALLOW 192.168.56.0/24
80/tcp ALLOW 192.168.56.0/24
443/tcp ALLOW 192.168.56.0/24
6443/tcp ALLOW 192.168.56.0/24
8472/udp ALLOW 192.168.56.0/24
10250/tcp ALLOW 192.168.56.0/24

blillo@app:~/pablo-asir-final/ansible$
```

Ahora, el de los workers, 12-ufw-workers.yml

```
- name: UFW workers (k3s agents)
  hosts: workers
  become: true
  vars_files:
    - ../group_vars/all/vars.yml
  tasks:
    - ufw: { rule: allow, port: "80", proto: tcp, src: "{{ lan_cidr }}" }
    - ufw: { rule: allow, port: "443", proto: tcp, src: "{{ lan_cidr }}" }
  }
    - ufw: { rule: allow, port: "8472", proto: udp, src: "{{ lan_cidr }}" }
  }
    - ufw: { rule: allow, port: "10250", proto: tcp, src: "{{ lan_cidr }}" }
  }
```

Y por ultimo, el hardening de la base de datos, 13-ufw-db.yml

```
- name: UFW DB (MariaDB externo)
  hosts: db
  become: true
  vars_files:
    - ../group_vars/all/vars.yml
  tasks:
    - ufw: { rule: allow, port: "3306", proto: tcp, src: "{{ master_ip }}" }

    - ufw:
        rule: allow
        port: "3306"
        proto: tcp
        src: "{{ item }}"
        loop: "{{ worker_ips }}"

    - ufw: { rule: allow, port: "3306", proto: tcp, src: "{{ backup_ip }}" }
```

Ahora, pasaremos a algo un poco más complicado, que es instalar MariaDB y las dependencias de mysql de Python, configurar el server para que escuche en 0.0.0.0 (que es como una wild card), crear la base de datos, crear el usuario y darle todos los permisos y añadir el usuario de backup para que también tenga acceso.

El playbook, 20-mariadb-incidencias.yml (incidencias porque la app es de gestión de incidencias, igual que la que tenemos en mi trabajo) sería el siguiente

```
- name: MariaDB (incidencias) en app-db
  hosts: db
  become: true
  vars_files:
    - ../group_vars/all/vars.yml
  tasks:
    - apt:
      name: [mariadb-server, python3-pymysql]
      update_cache: true
      state: present

    - service: { name: mariadb, state: started, enabled: true }

    - lineinfile:
      path: /etc/mysql/mariadb.conf.d/50-server.cnf
      regexp: '^bind-address'
      line: 'bind-address = 0.0.0.0'

    - service: { name: mariadb, state: restarted }

    - community.mysql.mysql_db:
      name: "{{ inc_db_name }}"
      state: present
      login_unix_socket: /var/run/mysqld/mysqld.sock

    - community.mysql.mysql_user:
      name: "{{ inc_db_user }}"
      password: "{{ inc_db_pass }}"
      priv: "{{ inc_db_name }}.*:ALL"
      host: "%"
      state: present
      login_unix_socket: /var/run/mysqld/mysqld.sock

    - community.mysql.mysql_user:
      name: "{{ backup_db_user }}"
      password: "{{ backup_db_pass }}"
      priv: "{{ inc_db_name }}.*:SELECT,SHOW VIEW,TRIGGER,LOCK TABLES"
      host: "%"
      state: present
      login_unix_socket: /var/run/mysqld/mysqld.sock
```

Nos Podemos fijar que está tirando de las variables de entorno que hemos creado anteriormente, lo cual hace todo más dinámico y escalable. Podemos comprobar inmediatamente que se ha instalado correctamente MariaDB

```
blillo@app:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.11.14 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-02-08 20:01:51 UTC; 42s ago
     Docs: man:mariadbd(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 19489 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /var/run/mysqld (code=exited, status=0/SUCCESS)
   Process: 19491 ExecStartPre=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=exited, status=0/SUCCESS)
   Process: 19493 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR= || VAR=/usr/bin/galera_recovery; [ $? -eq 0 ] && systemc
   Process: 19566 ExecStartPost=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=exited, status=0/SUCCESS)
   Process: 19569 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
  Main PID: 19553 (mariadbd)
    Status: "Taking your SQL requests now..."
     Tasks: 13 (limit: 30397)
    Memory: 79.0M (peak: 81.7M)
       CPU: 449ms
    CGroup: /system.slice/mariadb.service
            └─19553 /usr/sbin/mariadbd

feb 08 20:01:51 app-db mariadbd[19553]: 2026-02-08 20:01:51 0 [Note] InnoDB: File './ibtmp1' size is now 12.000MiB.
feb 08 20:01:51 app-db mariadbd[19553]: 2026-02-08 20:01:51 0 [Note] InnoDB: log sequence number 46980; transaction id 14
feb 08 20:01:51 app-db mariadbd[19553]: 2026-02-08 20:01:51 0 [Note] InnoDB: Loading buffer pool(s) from /var/lib/mysql/ib_buffer_pool
feb 08 20:01:51 app-db mariadbd[19553]: 2026-02-08 20:01:51 0 [Note] Plugin 'FEEDBACK' is disabled.
feb 08 20:01:51 app-db mariadbd[19553]: 2026-02-08 20:01:51 0 [Warning] You need to use --log-bin to make --expire-logs-days or --binlog-expire-log
feb 08 20:01:51 app-db mariadbd[19553]: 2026-02-08 20:01:51 0 [Note] Server socket created on IP: '0.0.0.0', port: '3306'.
feb 08 20:01:51 app-db mariadbd[19553]: 2026-02-08 20:01:51 0 [Note] InnoDB: Buffer pool(s) load completed at 260208 20:01:51
feb 08 20:01:51 app-db mariadbd[19553]: 2026-02-08 20:01:51 0 [Note] /usr/sbin/mariadbd: ready for connections.
feb 08 20:01:51 app-db mariadbd[19553]: Version: '10.11.14-MariaDB-0ubuntu0.24.04.1' socket: '/run/mysqld/mysqld.sock' port: 3306 Ubuntu 24.04
feb 08 20:01:51 app-db systemd[1]: Started mariadb.service - MariaDB 10.11.14 database server.
lines 1-28/28 (END)
```

Y también podemos comprobar que se ha creado la base de datos correctamente

```
blillo@app:~$ mysql -u inc_user -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 34
Server version: 10.11.14-MariaDB-0ubuntu0.24.04.1 Ubuntu 24.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> show tables
-> ;
ERROR 1046 (3D000): No database selected
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| incidencias |
| information_schema |
+-----+
```

En el siguiente paso, vamos a instalar k3s, que es una versión ligera de kubernetes, tanto en el master como en los workers, y los vamos a enlazar para que empiecen a trabajar y balanceen la carga cuando haga falta. El primer playbook sería 30-k3s-server.yml

```
- name: K3s server en app-master
  hosts: master
  become: true
  vars_files:
    - ../group_vars/all/vars.yml
  tasks:
    - apt: { name: curl, update_cache: true, state: present }

    - shell: |
        curl -sL https://get.k3s.io | INSTALL_K3S_EXEC="server --tls-san
        {{ master_ip }}" sh -
```

```

    args:
      creates: /usr/local/bin/k3s

- slurp:
    src: /var/lib/rancher/k3s/server/node-token
    register: token_b64

- set_fact:
    k3s_token: "{{ token_b64.content | b64decode | trim }}"

```

Y el siguiente sería 31-k3s-agents.yml

```

- name: K3s agents en workers
  hosts: workers
  become: true
  vars_files:
    - ../group_vars/all/vars.yml

  pre_tasks:
    - name: Leer token k3s desde el master
      slurp:
        src: /var/lib/rancher/k3s/server/node-token
      delegate_to: app-master
      run_once: true
      register: token_b64

    - name: Guardar token como variable
      set_fact:
        k3s_token: "{{ token_b64.content | b64decode | trim }}"

  tasks:
    - name: Instalar curl
      apt:
        name: curl
        update_cache: true
        state: present

    - name: Instalar K3s agent
      shell: |
        curl -sL https://get.k3s.io | K3S_URL="https://{{ master_ip
      }}:6443" K3S_TOKEN="{{ k3s_token }}" sh -
      args:
        creates: /usr/local/bin/k3s-agent

```

Tal como se puede ver, es sencillo, tiene algunas configuraciones específicas como el slurp o los args que son propias de kubernetes (rutas para que kubernetes pueda comprobar configuraciones) y además, crea un token de seguridad y se lo pasa a los workers-agents. Además, instala curl en caso que no esté instalado.

Una vez que hemos recibido el OK, podemos comprobar que los workers se han unido correctamente y están funcionando con el siguiente comando

```
blillo@app:~/pablo-asir-final/ansible$ sudo kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION      CONTAINER-RUNTIME
app-master           Ready     control-plane  4m1s  v1.34.3+k3s1  10.0.2.15     <none>         Ubuntu 24.04.3 LTS   6.8.0-100-generic   containerd://2.1.5-k3s1
app-worker1          Ready     <none>      28s   v1.34.3+k3s1  10.0.2.15     <none>         Ubuntu 24.04.3 LTS   6.8.0-100-generic   containerd://2.1.5-k3s1
app-worker2          Ready     <none>      28s   v1.34.3+k3s1  10.0.2.15     <none>         Ubuntu 24.04.3 LTS   6.8.0-100-generic   containerd://2.1.5-k3s1
blillo@app:~/pablo-asir-final/ansible$
```

El siguiente paso, que es el más largo, complicado y delicado, va a hacer lo siguiente: define que queremos 3 réplicas para el balanceo, crea un Namespace con el nombre de nuestra imagen en Dockerhub, configura kubernetes y define todo lo necesario para poder hacer peticiones a la base de datos, realiza el deployment con la imagen de Dockerhub de nuestra app, en master, en el puerto 5000, vincula esa imagen al puerto 80, que es el que está abierto, y configura Ingress para el master, que será nuestro reverse proxy. También instala la dependencia de Kubernetes en Python. Además, al final, comprueba que hay kubeconfig y que el deployment se ha completado de forma exitosa.

El playbook, 40-incidencias-k3s.yml sería el siguiente:

```
- name: Incidencias (Flask) en Kubernetes con replicas + Ingress
  hosts: master
  become: true

  vars_files:
    - ../group_vars/all/vars.yml

  pre_tasks:
    - name: Convertir replicas a int real
      set_fact:
        inc_replicas_int: "{{ inc_replicas | int }}"

    - name: Instalar dependencias Python para kubernetes.core.k8s (APT)
      apt:
        name:
          - python3-kubernetes
          - python3-yaml
        update_cache: true
        state: present

    - name: Verificar que existe el kubeconfig de k3s
      stat:
        path: "{{ kubeconfig_path }}"
      register: kubeconfig_stat

    - name: Fallar si no existe kubeconfig
      fail:
        msg: "No existe kubeconfig en {{ kubeconfig_path }}. ¿Has instalado k3s server y estás usando la ruta correcta?"
      when: not kubeconfig_stat.stat.exists
```

```

# Normalizamos imagen para evitar problemas de resolución
(k3s/containerd)

- name: Normalizar nombre de imagen (forzar docker.io)
  set_fact:
    inc_image_norm: >-
      {{
        (inc_image.startswith('docker.io/') | ternary(inc_image,
'docker.io/' ~ inc_image))
      }}

- name: Construir manifiesto Deployment Incidencias (YAML -> dict)
  con replicas INT real
  set_fact:
    inc_deployment_manifest: "{{ inc_deployment_manifest_yaml |
from_yaml }}"
  vars:
    inc_deployment_manifest_yaml: |
      apiVersion: apps/v1
      kind: Deployment
      metadata:
        name: {{ inc_app_name }}
        namespace: {{ inc_namespace }}
      spec:
        replicas: {{ inc_replicas_int }}
        selector:
          matchLabels:
            app: {{ inc_app_name }}
        template:
          metadata:
            labels:
              app: {{ inc_app_name }}
          spec:
            containers:
              - name: {{ inc_app_name }}
                image: {{ inc_image_norm }}
                imagePullPolicy: Always
                ports:
                  - containerPort: 5000
            envFrom:
              - secretRef:
                  name: inc-db-secret
            readinessProbe:
              httpGet:
                path: /health
                port: 5000
              initialDelaySeconds: 5
              periodSeconds: 10
            livenessProbe:

```

```

        httpGet:
          path: /health
          port: 5000
        initialDelaySeconds: 10
        periodSeconds: 15

tasks:
- name: Crear namespace
  kubernetes.core.k8s:
    kubeconfig: "{{ kubeconfig_path }}"
    state: present
    wait: true
    definition:
      apiVersion: v1
      kind: Namespace
      metadata:
        name: "{{ inc_namespace }}"

- name: Crear/actualizar Secret DB
  kubernetes.core.k8s:
    kubeconfig: "{{ kubeconfig_path }}"
    state: present
    wait: true
    definition:
      apiVersion: v1
      kind: Secret
      metadata:
        name: inc-db-secret
        namespace: "{{ inc_namespace }}"
      type: Opaque
      stringData:
        DB_HOST: "{{ inc_db_host }}"
        DB_PORT: "{{ inc_db_port }}"
        DB_NAME: "{{ inc_db_name }}"
        DB_USER: "{{ inc_db_user }}"
        DB_PASSWORD: "{{ inc_db_pass }}"

- name: Aplicar Deployment
  kubernetes.core.k8s:
    kubeconfig: "{{ kubeconfig_path }}"
    state: present
    wait: true
    definition: "{{ inc_deployment_manifest }}"

- name: Crear/actualizar Service
  kubernetes.core.k8s:
    kubeconfig: "{{ kubeconfig_path }}"
    state: present
    wait: true

```

```

definition:
  apiVersion: v1
  kind: Service
  metadata:
    name: "{{ inc_app_name }}"
    namespace: "{{ inc_namespace }}"
  spec:
    selector:
      app: "{{ inc_app_name }}"
    ports:
      - name: http
        port: 80
        targetPort: 5000

- name: Crear/actualizar Ingress
  kubernetes.core.k8s:
    kubeconfig: "{{ kubeconfig_path }}"
    state: present
    wait: true
    definition:
      apiVersion: networking.k8s.io/v1
      kind: Ingress
      metadata:
        name: "{{ inc_app_name }}-ingress"
        namespace: "{{ inc_namespace }}"
      spec:
        rules:
          - host: "{{ inc_host }}"
            http:
              paths:
                - path: /
                  pathType: Prefix
                  backend:
                    service:
                      name: "{{ inc_app_name }}"
                      port:
                        number: 80

- name: Reiniciar rollout para forzar pull de la imagen (si ya
  existía el deployment)
  shell: kubectl -n {{ inc_namespace }} rollout restart deploy/{{
  inc_app_name }}
  changed_when: false

- name: Esperar a que el deployment esté disponible
  shell: kubectl -n {{ inc_namespace }} rollout status deploy/{{
  inc_app_name }} --timeout=300s
  changed_when: false

```

```
- name: Mostrar estado (pods/svc/ingress)
  shell: |
    kubectl -n {{ inc_namespace }} get pods -o wide
    kubectl -n {{ inc_namespace }} get svc
    kubectl -n {{ inc_namespace }} get ingress
  changed_when: false
```

Por último, tenemos el playbook de la tarea con cron para hacer copias de seguridad de la base de datos. Este playbook va a definir varias cosas:

- Path del script y del log que se guardará posteriormente
- Instala dependencias por si no existen
- Crea un directorio de backups y le da permisos
- Crea archivos de credenciales secreto para poder acceder a la base de datos
- Crea el script de backup, que básicamente define variables básicas que ya están en vars.yml definidas (para poder escalar con más bases de datos en un futuro) como nombre de la base de datos y directorio de backup, días de retención del backup y el path del log, hace la copia de seguridad, la comprime, y guarda toda la información en un log
- Crea una tarea con cron con todos estos parámetros
- Comprueba que no ha habido errores

El playbook 50-backups-mariadb.yml sería:

```
- name: Configurar backups MariaDB comprimidos + cron en app-backup
  hosts: backup
  become: true

  vars_files:
    - ../group_vars/all/vars.yml

  vars:
    backup_script_path: /usr/local/bin/backup_mariadb.sh
    backup_log_path: /var/log/mariadb_backup.log

  tasks:
    - name: Instalar herramientas necesarias (cliente MariaDB + gzip)
      apt:
        name:
          - mariadb-client
          - gzip
        state: present
        update_cache: true

    - name: Crear directorio de backups
      file:
        path: "{{ backup_dir }}"
        state: directory
        owner: root
```

```

    group: root
    mode: "0755"

# Evita poner -pPASS en el comando (sale en ps/top). mysqldump leerá
credenciales de aquí.
- name: Crear /root/.my.cnf con credenciales de backup (0600)
  copy:
    dest: /root/.my.cnf
    owner: root
    group: root
    mode: "0600"
    content: |
      [client]
      user={{ backup_db_user }}
      password={{ backup_db_pass }}
      host={{ db_ip }}

- name: Crear script de backup (dump + gzip + retención)
  copy:
    dest: "{{ backup_script_path }}"
    owner: root
    group: root
    mode: "0755"
    content: |
      #!/usr/bin/env bash
      set -euo pipefail

      DB_NAME="{{ inc_db_name }}"
      BACKUP_DIR="{{ backup_dir }}"
      RETENTION_DAYS="{{ backup_retention_days }}"
      LOG="{{ backup_log_path }}"

      ts="$(date +%Y-%m-%d_%H-%M-%S)"
      out="{{ BACKUP_DIR}/${DB_NAME}_${ts}.sql.gz"
      tmp="{{ BACKUP_DIR}/${DB_NAME}_${ts}.sql.gz.tmp"

      mkdir -p "${BACKUP_DIR}"

      echo "[$(date +%F %T)] START backup ${DB_NAME}" >> "${LOG}"

      # mysqldump usa /root/.my.cnf (host/user/password)
      mysqldump --single-transaction --routines --triggers
"${DB_NAME}" \
    | gzip -c > "${tmp}"

      mv "${tmp}" "${out}"

# Retención

```

```

        find "${BACKUP_DIR}" -type f -name "*.sql.gz" -mtime
+ "${RETENTION_DAYS}" -print -delete >> "${LOG}" 2>&1 || true

        echo "[$(date +%F %T)] OK -> ${out}" >> "${LOG}"

- name: Programar cron diario a la hora configurada (backup_time
HH:MM)
  cron:
    name: "Backup MariaDB comprimido ({{ inc_db_name }})"
    user: root
    minute: "{{ (backup_time.split(':')[1]) | int }}"
    hour: "{{ (backup_time.split(':')[0]) | int }}"
    job: "{{ backup_script_path }} >> {{ backup_log_path }} 2>&1"

- name: Probar que el script existe y es ejecutable
  stat:
    path: "{{ backup_script_path }}"
    register: backup_script_stat

- name: Fallar si el script no está bien creado
  fail:
    msg: "No se ha creado correctamente {{ backup_script_path }}"
    when: not backup_script_stat.stat.exists or not
backup_script_stat.stat.executable

```

Para comprobar que se ha realizado todo correctamente, podemos ir a nuestra máquina app-backup y forzar el script a mano con este comando

```
sudo /usr/local/bin/backup_mariadb.sh
```

Y comprobamos que el backup y la tarea cron se ha creado con:

```
ls -lh /opt/backups/mariadb
sudo crontab -l
```

9. EVOLUCIÓN Y TRABAJO FUTURO

10. BIBLIOGRAFÍA